Ders 6 Genişletilmiş Notu: Maliyet Tahmin Modelleri, Proje Ekip ve Organizasyon Yapıları

Ders 6 Genişletilmiş Notu: Maliyet Tahmin Modelleri, Proje Ekip ve Organizasyon Yapıları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Proje Yönetimi Yalnızca Plan Değildir
- 2. Planda Şimdiye Kadar Gelinen Yer
- 3. Maliyet Tahmin Yöntemlerinin Tekrarı
- 4. COCOMO Modeli
- 5. Putnam Modeli
- 6. Yazılım Ölçümü
- 7. İşlev Noktası Hesabı
- 8. İşlev Noktasının Kullanıldığı Yerler
- 9. LOC ve İşlev Noktası Karşılaştırması
- 10. Maliyet Modellerinde Büyüklük Birimi
- 11. Araştırma Konuları: İnsan Davranışı ve Çevik Yaklaşımlar
- 12. Plandaki Son Adım: Organizasyon
- 13. Proje Ekip Yapıları
- 14. Proje Organizasyon Yapıları
- 15. Çevik Yaklaşımlarda Ekip Yapısı
- 16. Proje Zamanlaması: PERT, CPM ve Gantt
- 17. Yazılım Proje Planının İçeriği
- 18. Planlanan ile Gerçekleşenin İzlenmesi

1. Proje Yönetimi Yalnızca Plan Değildir

Hoca derse, proje yönetimini gerçekleştirmenin ne anlama geldiğini açıklayarak başladı. Proje yönetimini gerçekleştirmek, bir proje planı oluşturmakla bitmez. Seçilen uygulamanın geliştirme sürecinde bu planın nasıl takip edildiği de görülmelidir. Çalışma bir süre götürüldükten sonra planla karşılaştırılır. Bu, kontrol ve izleme aşamasıdır.

Bu nedenle proje yönetimi yol haritası tek bir plandan ibaret değildir. Hoca aşamaları şöyle saydı:

- Başlangıç aşaması: planı yapabilmek için gereken ilk çalışmalar.
- Planlama aşaması.
- Kontrol ve izleme aşaması ile hocanın “project running” dediği yürütme.
- Projenin kapatılması.

Bitmiş ve kaynak kodu hazır bir projenin planı üzerinden bu süreç izlenemez. Süreci görmek için planlanan ve geliştirme aşamasına biraz geçilmiş bir proje gerekir.

Soru: Sıfırdan bir projeyi planlayıp yalnızca planını ortaya koymak yeterli olur mu? **Cevap:** Biraz geliştirme aşamasına da geçmek gerekir. Çünkü yol haritası yalnızca plan değildir. Başlangıç, planlama, kontrol ve izleme, yürütme ve kapanışın hepsi süreçtedir.

Soru: Gerçek bir ürün mü bekleniyor, yoksa geliştirme üzerine konuşmak yeterli mi? **Cevap:** İlk evre olan gereksinim analizine girilmiş ve bir gereksinim analizi dokümanı oluşturulmuş olmalıdır. Sonra tasarımda hiç değilse küçük bir modülün başlangıcı yapılır. Önemli olan, en az iki kilometre taşı (milestone) geçildiğinde planlanandan sapmaların nasıl izlendiğidir. Riskler buna göre değiştirilir, plan da planlanan ile gerçekleşen arasındaki farklara göre revize edilir.

2. Planda Şimdiye Kadar Gelinen Yer

Hoca yeni konuya geçmeden önceki haftaları özetledi. Yazılım proje planı çerçevesinde önce risk tablosunun nasıl oluşturulduğu ve risklerin nasıl seçildiği konuşulmuştu. Plan sırasında üç şey üzerinde durulur: varsayımlar, kısıtlar ve projeyle ilişkili riskler.

Riskler, risk kategorileri üzerinden tanımlanmıştı. Varsayımlar ve kısıtlar için de benzer kategoriler oluşturulur. Böylece projeyle ilişkili varsayılan kurallar ve kararlar belirlenir. Kısıtlar ise proje elemanları, kaynaklar ve projedeki gereksinimler açısından ele alınır.

Sonra projenin adımları oluşturulur. İki hafta önce kilometre taşının ve aktivitelerin ne olduğu, aktiviteler arasındaki ilişkiler ve bunlar için MS Project'in nasıl kullanılacağı görülmüştü. Bunların üzerinden maliyet tahminine geçilir. Çünkü bütçeyi oluşturmak için yazılım maliyet tahmini gerekir.

3. Maliyet Tahmin Yöntemlerinin Tekrarı

Geçen hafta dört yöntem görülmüştü: bilirkişi takdiri, Delphi yöntemi, analiz yöntemi ve istatistik modeller.

3.1 Bilirkişi Takdiri ve Delphi

Delphi yöntemi, bilirkişi takdirinin çok uzmanlı hâlidir. Tek bir uzman yerine birden fazla uzman yönetilir. Sorular dağıtılır. Ortak karar alınana ya da en azından ortak karara yaklaşılana kadar süreç devam eder. Bu yöntem yalnız maliyet tahmininde değil, bir grubun karar aldığı bütün aşamalarda uygulanabilir.

3.2 Analiz Yöntemi

Analiz yönteminde proje bütün olarak değerlendirilmez. Alt parçalara bölünür ve bu parçaların maliyetleri üzerinde düşünülür. İki yol vardır:

- **Ürüne göre:** Ürünün alt modülleri ve alt parçaları belirlenir. Her birinin maliyeti, süresi, eforu ve bütçesi bulunmaya çalışılır. En küçük parçadan başlanır ve yukarı doğru çıkılır.
- **Üretime göre:** Üretimdeki safhalara göre bölünür. Planı yapmak ve incelemek için ne kadar süre, ne kadar efor ve kaç kişi gerekir? Geliştirmede tasarım, kodlama ve birim testi için kaç kişi gerekir? Test aşamasında ne kadar efor ve zaman gerekir?

3.3 İstatistik Modeller

İstatistik modeller, doğrudan ölçülemeyen bir değeri ölçülebilen bağımsız bir değişken yardımıyla kestirir. Yazılım proje tahmininde genellikle üslü fonksiyonlar kullanılır. Analiz yöntemi de bununla birlikte kullanılır, çünkü tahmin alt parçalara bölünerek yürütülür.

4. COCOMO Modeli

Bu alanda en yaygınlaşmış model COCOMO'dur (Constructive Cost Model). Barry Boehm tarafından oluşturulmuştur. Boehm, modeli aşama aşama ilerletmiştir: temel COCOMO, orta COCOMO ve ileri COCOMO.

> **Ders sonrası not:** Hoca Boehm'i "yazılım mühendisliğinin kurucusu" diye andı. Doğrusu büyük olasılıkla hocanın aynı cümlede söylediği ikinci nitelemedir: yazılım ekonomisinin başlangıcını yapan kişi. "Yazılım mühendisliği" terimi ve alanın kuruluşu büyük olasılıkla 1968 NATO konferansıyla ilişkilendirilir.

4.1 Temel COCOMO

İlk soru, iş hacminin neye göre bulunacağıdır. Bunun için önce bir proje büyüklüğü birimi seçilir. En basit birim satır sayısıdır. Boehm'in modelinde de birim, bin kod satırı (KLOC) olarak tanımlanmıştır.

Temel COCOMO'da iş hacmi şöyle hesaplanır:

$$E = a \cdot \text{KLOC}^{\,b}$$

a ve b katsayıları, Boehm'in çok sayıda proje üzerinde deneme yaparak hesapladığı değerlerdir. Temel COCOMO için bu katsayılar bellidir.

> **Ders sonrası not:** Hoca deneme sayısı için "64" dedi. Boehm'in COCOMO 81 veri kümesi büyük olasılıkla 63 projedir.

T değeri ise şu soruyu cevaplar: Bu kadar iş hacmindeki bir iş yükü ne kadar sürede etkin olarak gerçekleştirilebilir?

$$T = c \cdot E^{\,d}$$

Hoca burada önemli bir sezgi verdi. "Hızlı bitsin" diye bir sürü kişi yığarak bu iş yükü tamamlanamaz. Proje geliştirme, yani yazılım geliştirme işi belli bir süreçten geçmelidir. Süre, kişi sayısıyla keyfî biçimde kısaltılamaz.

Modelde sıra şöyledir:

1. Projenin büyüklüğü tahmin edilir: Ne kadar satırlık bir kod gerekir?
2. Bu kadarlık işin ne kadar iş gücüyle gerçekleştirileceği bulunur.
3. Bu kadar iş gücündeki işin ne kadar süreye yayılacağı bulunur.

> **Ders sonrası not:** Hoca "iki aşama var" dedi, ardından üç adım saydı. Büyük olasılıkla büyüklük tahminini girdi sayıp iş gücü ve süre hesaplarını kastetti. Akış üç adım olarak okunmalıdır.

4.2 Orta ve İleri COCOMO

Orta COCOMO'da temel modele bir düzeltme faktörü eklenir. Böylece biraz daha düzenlenmiş bir değer elde edilir. İleri COCOMO'da bu biraz daha geliştirilir. Hocaya göre temel COCOMO'dan hesaplansa bile, COCOMO çok eski olmasına rağmen proje hesapları için geliştirilmiş en uygun yöntemdir.

Tabii yeni yaklaşımlar da vardır. Hoca kendi yürüttüğü doktora çalışmalarından söz etti. Önce yapay sinir ağlarıyla bir maliyet tahmin modeli oluşturulmuştu. Çevik (agile) projeler geliştirilmeye başlanınca bu model çevik yaklaşımlara göre revize edildi.

4.3 Modelin Uygulanma Adımları

COCOMO bir projeye şu adımlarla uygulanır:

1. Üründeki bütün alt sistemler ve modüller tanımlanır.
2. Analiz yöntemiyle her modülün büyüklüğü tahmin edilir.
3. Orta COCOMO kullanılıyorsa, modül düzeyinde ürün, donanım ve personel özelliklerine göre düzeltme faktörleri alınır.
4. Her modül için, tablolardan alınan katsayılar ve faktörlerle iş hacmi ve proje süresi hesaplanır.
5. Modül sonuçlarından toplam iş gücü ve proje süresi bulunur.
6. Fayda-maliyet değerlendirmesi yapılır: Proje ne kadara mal olacak, faydası ne olacak? Bunun için duyarlılık analizleri gerçekleştirilir.
7. Planlama ve analiz giderleri eklenir.
8. Sonuç, Delphi yöntemiyle takdir edilen değerle karşılaştırılır ve farklılık giderilir.

4.4 Birleştirme Şaşkınlığı

Her modül için iş gücü ve süre hesaplandıktan sonra sonuçlar yukarı doğru birleştirilir. Bu sırada birleştirme şaşkınlığı denen olay yaşanır. Bizim, müşterinin ya da yönetimin kafasında bir ideal bitme süresi vardır. Hesaplanarak yukarı çıkıldığında ise bu süre iki buçuk sene gibi bir şey çıkabilir. Birleştirme şaşkınlığı budur.

Bunu düzeltmek için müşterinin ve yöneticinin kafasındaki süreye doğru gerilemeye çalışılır:

- Kaynaklar daha verimli kullanılır.
- Daha sıkışık bir çalışma düzenine geçilir.
- Gereksinimlerle örtüşen bazı şeyler revize edilir.

Delphi adımının rolü de buradadır. Beklentideki süre ve maliyet ile ideal hesaplanan iş hacmi arasındaki fark, Delphi yöntemiyle giderilmeye çalışılır.

4.5 Slayttaki Sayısal Örnek

Slaytta 10.000 satır kodlu bir yazılım için temel COCOMO hesabı vardı. Hoca slaytta üsleri normal yazdığını, 1,20 değerinin ve süre formülündeki değerin üs olması gerektiğini belirtti.

- **Verilen:** KLOC $= 10$, $a = 3{,}6$, $b = 1{,}20$.
- **Üs (ara adım eklendi):** $10^{1{,}20} \approx 15{,}85$.
- **İş hacmi (ara adım eklendi):** $E = 3{,}6 \times 15{,}85 \approx 57{,}06$ kişi-ay.

Hoca bu sonuçtan sürenin kaç ay olacağının ve kaç programcı gerektiğinin bulunduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Temel COCOMO'da proje türüne göre büyük olasılıkla üç katsayı takımı vardır: organik, yarı ayrık ve gömülü. Slayttaki 3,6 ve 1,20 büyük olasılıkla gömülü proje değerleridir.

5. Putnam Modeli

Başka bir model Putnam modelidir (altyazıda "Putnam-Rayli"). Burada da başlangıç KLOC'tur. İş gücü için KLOC, kullanılan teknolojiyi gösteren bir teknoloji katsayısı ve geliştirme süresi kullanılarak tahmin yürütülür. Hesaplama fonksiyonu COCOMO'nunkinden farklıdır.

> **Ek bilgi (derste anlatılmadı):** Putnam modeli büyük olasılıkla şu yazılım denklemine dayanır: $L = C_k \cdot K^{1/3} \cdot t_d^{4/3}$. Burada L büyüklük, C_k teknoloji katsayısı, K iş gücü, t_d geliştirme süresidir. Denklem iş gücü için çözülünce $K = L^3/(C_k^3\, t_d^4)$ çıkar. Adındaki Rayleigh, iş gücünün zamana dağılımını anlatan eğriden gelir.

Bu modellerin ortak mantığı şudur: Önce ne büyüklükte bir proje geliştirileceği tahmin edilir. Sonra buna ne kadar iş gücü ve ne kadar süre gerektiğine geçilir. İş gücü ve süre aşamalarının başlangıcı büyüklüktür. Bu da konuyu büyüklük ölçülerine, yani yazılım ölçümüne götürür.

6. Yazılım Ölçümü

Hoca yazılım mühendisliği tanımını hatırlattı. Bu tanımı geçen dönemki test dersinde de yapmıştı: Yazılım mühendisliği, yazılım ürününü oluşturmak için mühendislik yaklaşımı uygulamakla ilgili teknikler toplamıdır. Mühendislik yaklaşımında yönetme, maliyet hesabı, planlama, modelleme, analiz etme, tasarlama ve gerçekleştirme vardır. Bu adımların her birinde ölçme de vardır.

> **Hocanın vurgusu:** Yazılım mühendisliğinde ve bütün mühendislik yaklaşımlarında iki temel yetenek çok önemlidir: tasarlama ve ölçme. En önemli işlemler bunlardır.

Ölçme iki tiptir: doğrudan ve dolaylı.

6.1 Doğrudan Ölçme

Doğrudan ölçmenin örneği kod satırı (LOC, line of code) ve bin kod satırıdır (KLOC). En kolay ölçme şeklidir, çünkü satırlar sayılır. Ama çok objektif değildir. Etkin bir algoritma kullanılırsa kod azalır. Farklı bir kodlama dili kullanılırsa kod artabilir.

Bu iki etkinin durumu farklıdır. Diller arası fark için dönüşüm tablosu vardır: hangi dildeki kaç satırın başka bir dildeki kaç satıra karşılık geldiği bilinir. Bu yüzden bu fark üzerinde çok durulmayabilir. Algoritmanın yapısına ve etkinliğine göre değişen satır sayısı ise LOC'un zayıf tarafıdır.

6.2 Dolaylı Ölçme

Dolaylı ölçmede yazılımın özelliklerinden yola çıkılır: işlev, nitelik, karmaşıklık, etkinlik, güvenilirlik. Özellikle kaç belge, kaç dosya ve kaç ekran olduğu gibi sayılar kullanılır. Dolaylı ölçümün örneği işlev noktasıdır (Function Point, FP).

7. İşlev Noktası Hesabı

7.1 Ham İşlev Noktası

İşlev noktası bir tablo üzerinden hesaplanır. Tabloda beş öğe sayılır:

- Kullanıcının yazılıma giriş (girdi) sayısı.
- Aldığı çıktı sayısı.
- Kullanıcının sorgulama sayısı.
- Dosya sayısı.
- Dış arabirim sayısı.

Her öğenin kaç farklı tipi olduğu sayılır ve tabloya yazılır. Hoca örnek olarak "kullanıcının girdi sayısı diyelim ki 15" dedi.

Tablonun sağ tarafında ağırlık faktörü vardır. Sayının basit, orta veya karmaşık aralıklarından hangisine girdiği başka bir tablodan bakılır. Ağırlık faktörü, o sayının işlev noktasını ne kadar etkileyeceğini gösterir. Girdi satırı için hocanın söylediği ağırlıklar basitse 3, ortaysa 4'tür. Örneğin:

- Girdi sayısı 15 ve basit aralıkta ise (ara adım eklendi): $15 \times 3 = 45$.
- Girdi sayısı 15 ve orta aralıkta ise (ara adım eklendi): $15 \times 4 = 60$.

Her satır için sayı ile ağırlık faktörü çarpılarak o satırın işlev noktası bulunur. Bütün satırların toplamı ham işlev noktasıdır. Tabloda "sayı toplamı" diye geçer.

7.2 Düzeltilmiş İşlev Noktası

Ham değer doğrudan kullanılmaz. Beş sayı, çalışan yazılımda takip edilebilecek değerlerdir. Hazırlanan yazılımın özellikleri de sonucu etkiler. Bu yüzden 14 soru sorulur ve cevaplardan bir karmaşıklık düzeltmesi yapılır:

$$FP = \text{sayı toplamı} \times \left(0{,}65 + 0{,}01 \sum_{i=1}^{14} F_i \right)$$

Her F_i, bir sorunun 0 ile 5 arasında verilen cevabıdır. Soru uygulama için çok önemliyse değer yüksek verilir. Hocanın saydığı örnek sorular şunlardır:

- Güvenli yedekleme ve geri yükleme (backup recovery) önemli mi?
- İletişim altyapısı gerekli mi?
- Dağıtılmış özellikler var mı?
- Performans kritik mi?
- Sistem yükü fazla mı?
- Çevrim içi veri girişi var mı, yani sürekli online veri girişi yapılıyor mu?

Slayttaki örnekte iki soruya 5 verilmişti. Çevrim içi veri girişi 5, yedekleme ve geri yükleme 4 idi. 14 sorunun toplamı $\sum F_i = 32$ çıkıyordu.

Bu değer formüle yerleştirilir (ara adım eklendi):

$$0{,}65 + 0{,}01 \times 32 = 0{,}65 + 0{,}32 = 0{,}97$$

Yani bu örnekte düzeltilmiş işlev noktası, sayı toplamının $0{,}97$ katıdır.

> **Ek bilgi (derste anlatılmadı):** 14 sorunun her biri 0 ile 5 arasında olduğu için toplam 0 ile 70 arasında değişir. Bu yüzden düzeltme çarpanı $0{,}65$ ile $1{,}35$ arasında kalır. Yani yazılımın özellikleri ham değeri en çok yüzde 35 azaltır ya da artırır.

8. İşlev Noktasının Kullanıldığı Yerler

Düzeltilmiş işlev noktası, LOC ve KLOC gibi kullanılır. Hepsinde soruların cevabı için girdi olur:

- **Verimlilik:** Bu büyüklükte bir iş kaç kişiyle, kaç adam-ayda geliştirilir?
- **Kalite:** Bir ayda şu kadar hata çıktıysa hata bölü işlev noktası hesaplanır.
- **Gider:** Toplam gider bölü işlev noktası. Bir işlev noktası kaça mal oldu?
- **Belgeleme:** Bu büyüklükteki bir yazılım için ne kadar belge oluşturuldu?

9. LOC ve İşlev Noktası Karşılaştırması

Hoca iki ölçüyü şöyle karşılaştırdı:

Ölçüt	LOC	İşlev noktası
Programlama dili	Dile bağlıdır (dönüşüm tablosu olsa da)	Dilden bağımsızdır
Nasıl bulunur	Satırlar doğrudan sayılır	Dosya, ekran, girdi ve çıktı sayılarından dışarıdan hesaplanır
Fiziksel ölçü mü	Evet. En büyük faydası budur	Hayır. Fiziksel ölçülerin işlenmesiyle bulunur
Nesnellik	Satır sayımı nettir, ama algoritmaya göre değişir	Sübjektif bir değerdir
Veri toplama	Kolaydır	Daha güçtür
Zayıf yön	İyi tasarlanmış kısa yazılımları yeterince değerlendiremez	Sübjektiftir ve verisi zor toplanır

İyi tasarlanmış bir yazılımda algoritma daha etkin olduğu için satır sayısı az olabilir. LOC böyle kısa yazılımları değerlendirmek için iyi bir yöntem olmaz. Sonuç olarak iki ölçüden çalışmaya hangisi uygunsa o kullanılır.

> **Ders sonrası not:** Hoca "işlev noktası işlemsel olmayan dillerdeki yazılımlara kolayca uyarlanamaz" dedi ve görsel dilleri, dördüncü kuşak teknikleri örnek verdi. Bu zayıflık 5. dersin slaytında LOC sütunundaydı. Hoca büyük olasılıkla LOC'u kastetti. İşlev noktası dilden bağımsız olduğu için bu sorun LOC için geçerlidir.

10. Maliyet Modellerinde Büyüklük Birimi

> **Hocanın vurgusu:** Yazılım ölçümü (software metrics) hem yazılım öğretisinin önemli konularından biridir, hem de bu dersin temel konularındandır.

İstatistik modellerdeki iki tahmin yöntemi görüldü: ilk kullanılan COCOMO ve Putnam modeli. İkisinin de baz aldığı proje büyüklüğü birimi KLOC'tur. KLOC yerine işlev noktası kullanan yöntemler de vardır. Bunlardan biri Albrecht yöntemi, yani fonksiyon noktası maliyet yöntemidir.

Mantık aynıdır. COCOMO'da iş hacmi KLOC büyüklüğünden bulunur. Birçok başka istatistik model, farklı formüller kullansa da başlangıç noktası olarak işlev noktasıyla ölçülen büyüklüğü alır. Sonra iş gücüne, iş gücünden de zamana geçilir.

11. Araştırma Konuları: İnsan Davranışı ve Çevik Yaklaşımlar

Aradan sonra hoca, makale önerileri üzerinden iki araştırma yönünden söz etti.

İlki yazılımcı hatasının insan-bilgisayar etkileşimi ve insan davranışı açısından değerlendirilmesidir. Hocanın grubunun da son zamanlarda ilgilendiği bir konudur. Bu yönde kod yazan kişilerin kişilik sınıflaması ile ilişkisi incelenmiştir. Nesneye yönelik yaklaşım dersindeki öğrencilerin kişilik testleri yapılarak hangi konularda başarılı oldukları da araştırılmıştır.

Bu konudaki bir makale IEEE Transactions on Software Engineering'de yayımlanmıştı. Hoca konuyu kalite dersiyle de ilişkilendirdi. Bir başka makale, insan kişilik ilişkileri üzerine bir konferansta yayımlanmıştı ve ekip yapısının nasıl kurulması gerektiğini soruyordu. Çünkü hataların ortaya çıkması kişilik özellikleriyle ilişkilidir, bu da ekip yapısının nasıl kurulacağı sorusuna götürür.

İkinci yön çevik yaklaşımlardır. Birçok öğrenci çalıştığı yerde çevik yaklaşım kullandığı için öneriler çakışmış, Scrum ve çevik yaklaşımların derlemesi gibi tekrarlı konular gelmişti. Hoca temel bilgilerin dışına çıkılmasını istedi: Konu farklılaştırılmalı, özelleştirilmeli ve daha ayrıntılı yönlerine doğru incelenmelidir. Yeni açılımlar aranmalıdır.

12. Plandaki Son Adım: Organizasyon

Yazılım proje planında şimdiye kadar yapılanlar şunlardır:

- Risk analizi yapıldı.
- Varsayımlar ve kısıtlar belirlendi.
- Aktiviteler ve aralarındaki ilişkiler belirlendi.
- Gantt diyagramı ile zaman planı çıkarıldı. Bunun için PERT'ten yararlanıldı, projenin kritik yolu ve kritik görevler bulundu.
- Her aktivitenin süresi ve büyüklüğü, COCOMO'daki gibi maliyet tahmin yöntemleriyle birleştirildi.
- Bunun için önce projenin büyüklüğü, geçmiş projelerden alınan sonuçlara göre tahmin edildi.
- Böylece bütçe hazırlandı: ne kadar zaman, kaç kişi, hangi yetenekte kişiler.

Geriye organizasyon kalır: Nasıl bir proje ekip yapısı ve nasıl bir organizasyon oluşturulacak?

13. Proje Ekip Yapıları

Üç temel proje ekip yapısı vardır: demokratik, şeflik ve hiyerarşik.

13.1 Demokratik Ekip Yapısı

Bütün üyeler eşittir ve aynı yetkiye sahiptir. Yazılım geliştirme amaçları ve yöntemleri üyelerin birbiriyle anlaşması yoluyla saptanır. Kararlar birleştirilerek alınır. Olsa olsa herkesle iletişimi sağlamak üzere bir koordinatör seçilebilir.

Aynı seviyedeki birçok kişinin bir araya geldiği bu yapı 8-10 kişiden fazla olamaz. İşler biraz daha yavaş yürür, çünkü herkes konuşur. Bir kişi karar verip herkesin onu uygulaması söz konusu değildir.

13.2 Şeflik Ekip Yapısı

Ürün tasarımı, geliştirilmesi ve diğer teknik konularda şef karar verir. Görev dağıtımını da şef yapar. Yaklaşık beş kişilik bir ekip düşünülür. Hocanın örneği küçük bir yazılım evidir: Deneyimli bir yazılım veya bilgisayar mühendisi şirket kurmuştur, altında tamamen ona bağlı 5-7 çalışan vardır.

Kararlar bir kişi tarafından alındığı için iş daha hızlı yürüyebilir. Ama kararların doğruluğu ve tutarlılığı şefin yeteneğine bağlıdır. Kalabalık arttıkça da iş düzgün ilerlemez.

> **Hocanın vurgusu:** Hoca on kişiye kadar şeflik sisteminin hâlâ kullanılabileceğini söyledi. Ondan fazla çalışanı olan iş yerleri ve proje ekipleri için şeflik sistemi önerilmez. Onlar için hiyerarşik sistem önerilir.

13.3 Hiyerarşik Ekip Yapısı

Bir proje yöneticisine bağlı birkaç ekip oluşturulur. Üç katman vardır: proje yöneticisi, onun altında çalışan şefler ve diğer işleri gerçekleştiren çalışanlar.

- **Proje yöneticisi:** Görevleri belirler, çalışmaları izler ve denetler, iş yüklerini dengeler. Ama her işe kendisi karar vermez.
- **Ekip şefleri:** Kendilerine ayrılan alt sistem veya modülün geliştirilmesinden sorumludur.

Altta, her uygulama ve iş grubuyla ilişkili şefler, karar vericiler ya da bölüm yöneticileri bulunur. Kalabalık bir proje ofisi için en uygun sistem budur.

13.4 Karşılaştırma

Yapı	Karar	Kişi sayısı	Güçlü yan	Zayıf yan
Demokratik	Uzlaşmayla	En çok 8-10	Herkes eşit, kararlar birleştirilir	İşler yavaş yürür
Şeflik	Şef verir	Yaklaşık 5, en çok 10	Hızlıdır	Kararların doğruluğu şefin yeteneğine bağlıdır
Hiyerarşik	Yönetici ve ekip şefleri	10’dan büyük ekipler	Kalabalık ekipler için uygundur	Derste belirtilmedi

14. Proje Organizasyon Yapıları

Hiyerarşik yapının nasıl kurulacağı sorusu, proje organizasyon yapılarına götürür. Üç seçenek vardır: fonksiyonel yapı, proje takımları yapısı ve matris yapı.

Her organizasyon tipinin kendine göre faydası vardır. Her ortama uygun tek bir tip olduğu söylenemez. Seçim birçok faktöre bağlıdır: yapılacak iş, çalışılacak ortam, kurumun ve organizasyonun amaçları.

14.1 Fonksiyonel Organizasyon

Çalışanlar uzmanlık alanlarına göre ayrılır ve farklı bölümlere yerleştirilir. Proje bazında düşünülürse proje, gerekli uzmanlık gruplarına göre bölümlere ayrılır. Her grubun başı projenin kendi kısmından sorumludur.

Hoca bir noktayı özellikle ayırdı. Burada söz konusu olan, şirket çalışanlarının kadrosal olarak bağlı olduğu birimler değildir. Çok büyük bir projenin kendi sistem analistleri, kendi programlama, pazarlama ve bakım sorumluları gibi alt ekiplere ayrılmasıdır. Slaytta proje yöneticisinin altında sistem analiz bölümünün yöneticisi, bakım sorumlusu gibi birimler vardı.

```
Proje Yöneticisi
├── Sistem Analizi Bölümü
├── Programlama Bölümü
├── Pazarlama Bölümü
└── Bakım Bölümü
```

Bu yapı ancak çok büyük organizasyonlarda kullanılır. Küçük bir ekip için sağlıklı ve gerçekçi değildir. Küçük bir proje veya şirkette hantal kalır.

14.2 Proje Organizasyonu

Sistemden istenen iş ana parçalara ayrılır. Her parçanın gerçekleştirilmesi için gereken bütün uzmanlık alanlarından kişiler ilgili birime atanır. Örneğin genel uygulamalar sistemi içinde muhasebe modülü için ayrı bir organizasyon birimi kurulur.

Slayttaki örnekte proje yöneticisine bağlı destek grupları ve modül ekipleri vardı:

```
Proje Yöneticisi
├── Destek: bakım yöneticisi, kalite sorumlusu, satış sonrası destek uzmanı
├── Stok Kontrol Ekip Lideri ── sistem analisti, programcı
├── Finans Modülü Ekip Lideri ── sistem analisti, programcılar
├── Yönetim Bilgi Sistemleri Ekibi ── sistem analisti, programcılar
└── Müşteri Ekranları Ekibi
```

Yazılımın her farklı modülü için ayrı bir ekip kurulur. Ekipler bazen paralel, bazen birbirini izleyerek çalışır, ama her birinin ekibi ayrıdır. Böylece projenin gerçekleştirdiği her alt işin bir sorumlusu olur. Hoca slaytta yalnız programcı ve sistem analisti yazılmasının minimalist olduğunu, gerçekte yazılım mühendisi, tester gibi daha ayrıntılı rollerin de tanımlanabileceğini söyledi.

14.3 Matris Organizasyon

Birçok firmada uygulanan yapıdır. Raporlama ve yetki açısından klasik organizasyonlardaki ağaç yapısından farklı olarak bir tablo yapısı şeklinde çalışır. Temel özelliği, yetkinin birden fazla kişiye dağıtılmış olmasıdır. Proje organizasyonu ile fonksiyonel organizasyonun üst üste konup birleştirilmiş hâli gibi düşünülebilir.

Projede çalışan uzmanlar tek bir kişiye raporlamak yerine, konunun sorumlusuna göre iki kişiden birine bilgi aktarır.

Slayttaki şekilde iki tür yönetici vardı:

- **Sürekli yöneticiler:** Yazılım mühendisliği lideri, sistem analistleri lideri, programcılar lideri. Genel müdüre bağlıdırlar ve o gruptaki herkesin işini sürekli kontrol ederler.
- **Proje yöneticileri:** A projesi, B projesi gibi projelerin yöneticileri. Projedeki görevleri kontrol ederler.

	A Projesi Yöneticisi	B Projesi Yöneticisi
Yazılım Müh. Lideri	yazılım mühendisi	yazılım mühendisi
Sistem Analistleri Lid.	sistem analisti	sistem analisti
Programcılar Lideri	programcı	programcı

A projesine atanan bir programcıyı hem programcılar lideri hem de A projesinin yöneticisi kontrol eder. Slayttaki gri çizgi proje yöneticisinden programcıya gelen bu kontrolü gösteriyordu. B projesindeki sistem analisti için de aynısı geçerlidir. Hangi tür iş için kimden izin alınacağı ve görev atamada programcılar liderinin nasıl davranacağı bu yapıda tanımlanır.

14.4 Üç Yapının Özeti

Hoca dersin sonuna doğru üç yapıyı yan yana getirdi:

- **Fonksiyonel:** Proje yöneticisi, ayrı bir sistem analisti grubu, ayrı bir programcılar grubu. Yapılan her iş için ayrı bir grup.
- **Proje:** Proje yöneticisi, bakım yöneticisi, stok kontrol takımı gibi. Her modül için ayrı bir ekip.
- **Matris:** Her ikisi birlikte, tablo şeklinde. Kurumun yöneticileri ve bunlara bağlı çalışanlar tanımlıdır. Ayrıca projede sorumluluklar vardır: A projesinde sistem analistinin, yazılım geliştiricilerin, web tasarım uzmanlarının yaptığı işler ayrılır.

15. Çevik Yaklaşımlarda Ekip Yapısı

Hoca slayt kullanmadan, sorulmamış ama akla gelebilecek bir soruya cevap verdi: Bunlar klasik organizasyon ve ekip yapılarıdır. Oysa küçük büyük bütün şirketlerde çevik yaklaşımlar

kullanılıyor. O zaman ne olacak?

Scrum'a bakıldığında herkes birbirini kontrol eder ve bir proje yöneticisi yoktur. Şirkette yine de programcıları yöneten, başarı grafiklerini isteyen, günlük izinlerini veren bir yazılım geliştiriciler grubu olabilir. Ama projedeki yapılanma ne matrise, ne proje organizasyonuna, ne de fonksiyonel organizasyona benzer. Ekip yapıları çevik yaklaşıma göre oluşturulur.

Çevik ekipte herkes birbirinin yaptığından haberdardır. Kimse "ben programcıyım, gerisini bilmiyorum" demez. Projenin içinde "şu fonksiyonu yerine getirenler" ya da "şu alt modülde çalışanlar" diye alt başlık açılmaz. Ürün daha genel bir grubun ürünü olarak ortaya çıkar.

16. Proje Zamanlaması: PERT, CPM ve Gantt

Proje zamanlamasında durak noktalarında onaya sunmak gerekir. Yazılım geliştirme projesinin onay noktaları ve adımları kesin tarihler hâlinde planlanmalı, süre bitiminde de teslim edilmelidir.

Zamanlama PERT, CPM ve Gantt çizelgesiyle gösterilir. PERT ve CPM proje süresini belirleyen kritik yolu bulmaya yarar. PERT ayrıca istatistik fonksiyonlar kullanarak her işlem için en olası zamanı kestirir. CPM'de bu yoktur, çünkü CPM belirlilik hâlinde çalışır: Her işin ne kadar süreceği önceden belirlenmiştir.

Yazılım geliştirmede ise durum böyle değildir. Planlama tahmin edilenden uzun sürebilir, gereksinim analizi de öyle. PERT adımları belirli bir işlemin başlama ve bitirme aralığını hesaplamayı sağlar.

> **Ders sonrası not:** Hoca PERT ve CPM için "birbirinin yerine geçen, aynı işi yapıyorlar" dedi, ardından farklarını anlattı. Doğrusu: İkisi de kritik yolu bulur, ama birbirinin tam yerine geçmez. CPM süreleri kesin kabul eder, PERT belirsiz süreler için üç tahminle çalışır (5. ders).

17. Yazılım Proje Planının İçeriği

Yazılım geliştirme planı, göreve başlarken ne yapılacağını, kaynakların neler olduğunu ve hangi görevlerin hangi sürede tamamlanacağını belirlemek için oluşturulur. Yazılım mühendisinin bilgi ve deneyimine dayanarak hazırlanır. Görevin yerine getirilmesi sırasında plan yeniden gözden geçirilir ve üzerinde her zaman düzeltme yapılabilir.

Proje planında şunlar bulunur:

1. Projenin kapsamı.
2. Kullanılacak yazılım geliştirme süreç modeli.
3. Risk analizi.
4. Alt aktiviteler, yani iş kırılım yapısı (work breakdown structure, WBS).
5. Aktiviteler arasındaki ilişkiler.
6. İş gücünün belirlenmesi. Bu, maliyet tahmin yöntemleriyle yapılır.
7. Görev ataması.
8. Bütçe.
9. Kalite planı.
10. Yazılım gereksinim spesifikasyonu (SRS).

Hoca sıraya dikkat çekti: Önce süreç modeli seçilir, sonra WBS'ye geçilir.

Görev atamasından daha önce çok bahsedilmemişti. İş gücü maliyet tahminiyle belirlenir. Ama diyelim ki üç programcı var: Görev hangisine atanacak? Önce rastgele atanabilir. Sonra bir kişinin üzerinde çok yoğunluk varsa düzenleme yapılır. Dengeli görev ataması için endüstri mühendisliğinin bazı programları ve bu konuda çalışmalar vardır.

18. Planlanan ile Gerçekleşenin İzlenmesi

Hoca, dersin başındaki konuya dönerek süreci bir daha anlattı. Tamamen bitmiş bir projenin planı bu süreci göstermez. Kafada olan ya da üzerinde çalışılıp prototip üretilen bir projenin planı çıkarılır. Akış şöyledir:

1. Proje planı çıkarılır.
2. Geliştirmeye devam edilerek gereksinim analizi yapılır.
3. Planlanan yol ile gerçekleşen yol arasında fark olup olmadığı kontrol edilir.
4. Gereksinim analizi raporu oluşturulur.
5. Sonraki aşama olarak yazılım programlanır. Gerçekleşen durum ile ilk hazırlanan plan arasındaki fark yine takip edilir.

Gantt diyagramında planlanan ile gerçekleşeni karşılaştırma özelliği vardır. Ayarlarına bakılarak denenebilir. Hoca planın nasıl yapılacağıyla ilgili bir örnek çalışmayı sonraki hafta göstermeyi önerdi.

Dersin Toparlaması

Bu derste proje planının maliyet ve organizasyon kısımları tamamlandı. Maliyet tahmininin başlangıcı büyüklüktür: Büyüklük KLOC ile doğrudan ya da işlev noktası ile dolaylı ölçülür. Sonra COCOMO, Putnam veya Albrecht gibi modellerle iş gücüne, iş gücünden süreye geçilir. Modüllerden yukarı toplanan sonuç beklentiden uzun çıkabilir. Birleştirme şaşkınlığı denen bu fark, kaynak kullanımı, çalışma düzeni ve gereksinimler gözden geçirilerek ve Delphi ile karşılaştırılarak kapatılmaya çalışılır.

Organizasyon tarafında ekip büyüklüğü ekip yapısını belirler: Küçük ekipler için demokratik veya şeflik, ondan büyükler için hiyerarşik yapı uygundur. Hiyerarşik yapı da işe, ortama ve kurumun amaçlarına göre fonksiyonel, proje veya matris biçiminde kurulur. Çevik yaklaşımlar bu klasik kalıplara uymaz. Son olarak planın içeriği sıralandı ve planın bir kez yazılıp bırakılmadığı, kilometre taşlarında gerçekleşenle karşılaştırılıp revize edildiği vurgulandı. Sonraki hafta plan için bir örnek çalışmaya bakılabilecek.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.